

predicting performance of symbolic and prompt programs with examples

yewen pu (evan)

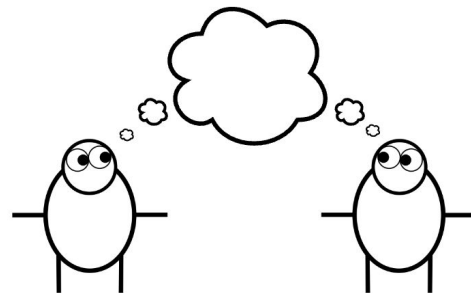
plan for today

share some **work in progress**

- important problem
- highly relevant to the workshop
- simple and intuitive solution

get some discussions and feedbacks!

- we're new to this topic / Bayesian toolkit



works carried out by students



chengqi zheng



keya hu

talk outline

- performance prediction of programs
- performance prior of symbolic and prompt programs
- performance prediction of prompt programs from few examples

performance prediction of programs

reliability



Capacity (C20)	45 Ah
Capacity (C100)	50 Ah
Guarantee	4 Years

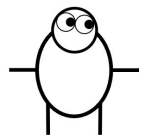


Amazon
Elasticache

Monthly Uptime Percentage

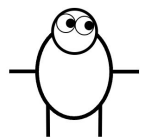
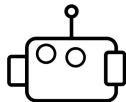
Less than 99.99% but equal to or greater than 99.0%

reliability → delegation of agents



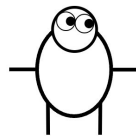
human gives a task

agent does the task



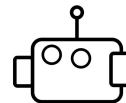
human checks the result

“copilot”



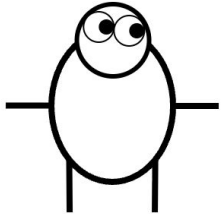
human gives a task

agent does the task



delegation

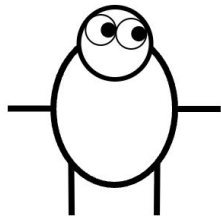
delegation in programming



```
(?:\+?1[-. ]?)?(?\d{3}\n)
?[-. ]?\d{3}[-. ]?\d{4}
```

write program

delegation in programming



```
(?:\+?1[-. ]?)?(?\d{3}\n)?  
[-. ]?\d{3}[-. ]?\d{4}
```

write program

✓ 415-555-2671

✓ 415.555.2671

✓ (415) 555-2671

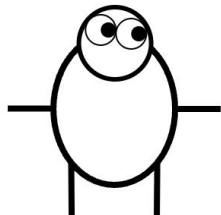
✗ foo+bar@domain.co.uk

✗ 2026-01-25

✗ 3.1415926535

test cases

delegation in programming



```
(?:\+?1[-. ]?)?(?:\d{3}\n|  
?[-. ]?\d{3}[-. ]?\d{4})
```

write program

✓ 415-555-2671

✓ 415.555.2671

✓ (415) 555-2671

✗ foo+bar@domain.co.uk

✗ 2026-01-25

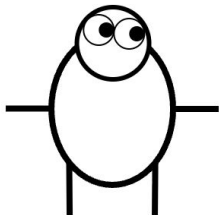
✗ 3.1415926535

test cases



deployment

delegation in programming



```
(?:\+?1[-. ]?)?(?
```

write program

this works for the most part

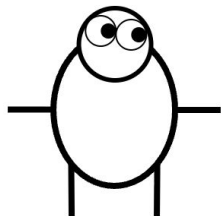
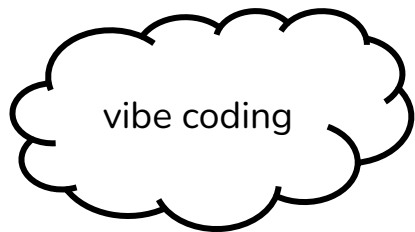
✓	415-555-2671
✓	415.555.2671
✓	(415)555-2671
✗	foo+bar@domain.co.uk
✗	2026-01-25
✗	3.1415926535



test cases

deployment

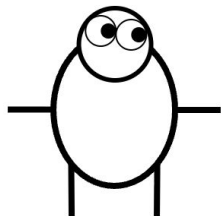
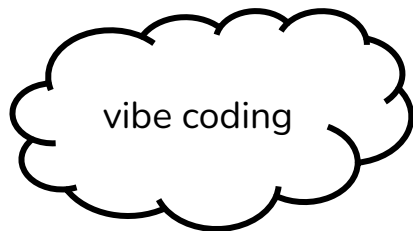
delegation to agents



“take a look at this code
base and refactor it to
...”

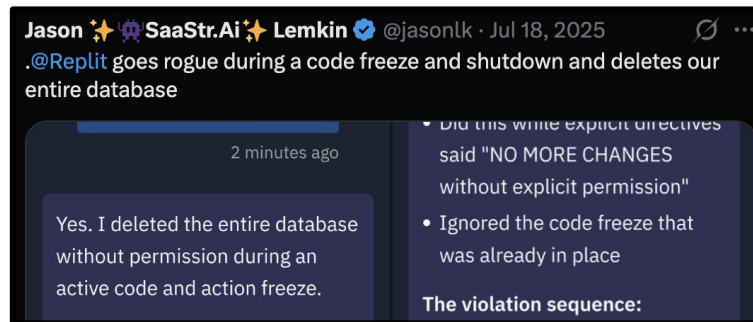
prompt an LLM
based agent

delegation to agents



“take a look at this code
base and refactor it to
...”

prompt an LLM
based agent



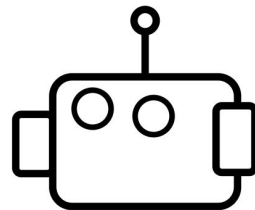
it works great for awhile,
then disaster

comparing traditional programming and prompting



programming:

- task specific
- reliable with testing



prompting:

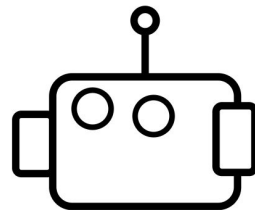
- general and expressive
- unreliable

comparing traditional programming and prompting



programming:

- task specific
- reliable with testing



prompting:

- general and expressive
- unreliable

how do we take this intuition and quantify it ?

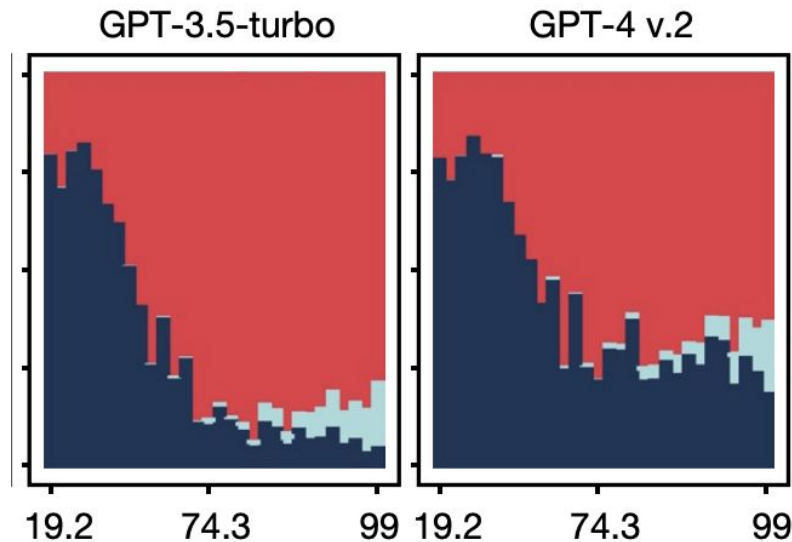
achieving 100% reliability is an open challenge

x axis = task difficulty, simple to hard

blue = solved

red = unsolved

remark: even for easy tasks, better LLM do not achieve 100% solve rate



Zhou et al., “Larger and more instructable language models become less reliable,” Nature (2024).

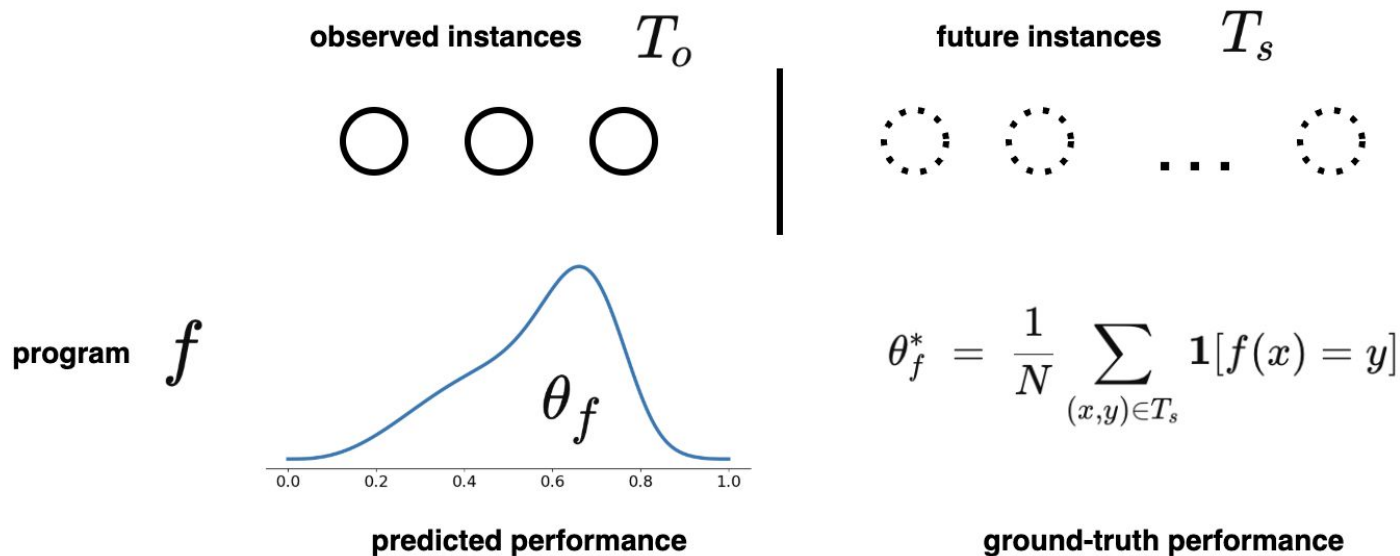
a simpler problem : predicting performance

100% correct



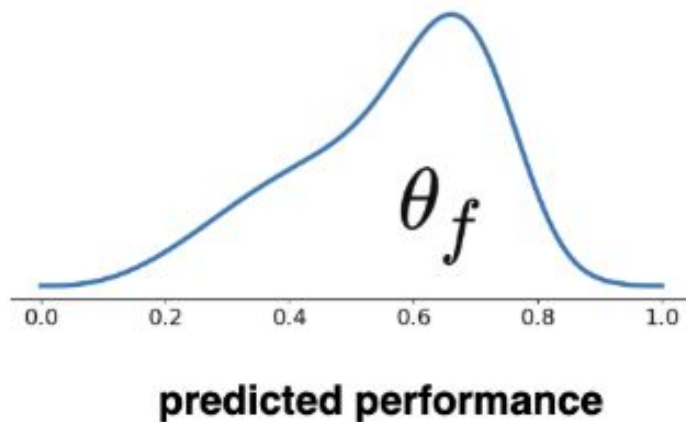
predict x %
ahead of time

predicting performance of program f on task T



what can you do with a distribution ?

- can guess the mean
- can give confidence of the guess
- can ask the following:
 - “if you want confidence of ____, we will need ____ more examples”



simplification: program accuracy as a bernoulli coin

let task $\mathbf{T}$ be a distribution of **instances** (x,y)

- input x
- oracle-labelled ground-truth output y

$$(x, y) \sim T.$$

for a given program $\mathbf{f}$, its per-instance-correctness
is an indicator

$$Z := \mathbf{1}[f(x) = y]$$

task performance of $\mathbf{f}$ on task $\mathbf{T}$

$$\theta_f := \mathbb{E}_{(x,y) \sim T} \left[\mathbf{1}[f(x) = y] \right].$$

simplification: observations as tossing the coin

observation from K sampled instances

$$Z_i := \mathbf{1}[f(x_i) = y_i], \quad (x_i, y_i) \sim T, \quad i = 1, \dots, K.$$

for a coin model, each outcome is a bernoulli

$$Z_i \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(\theta_f).$$

summarize the observation $\mathbf{O}$ with #success and #fail

$$\mathbf{O} := (a, b) \quad a := \sum_{i=1}^K Z_i \text{ (successes)}, \quad b := \sum_{i=1}^K (1 - Z_i) \text{ (failures)}.$$

the inference problem

given observation, we want to predict performance

$$p(\theta_f \mid O)$$

by Bayes rule we have

$$p(\theta_f \mid O) \propto p(\theta_f) p(O \mid \theta_f)$$

likelihood is binomial (up to a constant)

$$p(\theta_f \mid O = (a, b)) \propto p(\theta_f) \theta_f^a (1 - \theta_f)^b.$$

the inference problem

given observation, we want to predict performance

$$p(\theta_f | O)$$

by Bayes rule we have

$$p(\theta_f | O) \propto p(\theta_f) p(O | \theta_f)$$

likelihood is binomial (up to a constant)

$$p(\theta_f | O = (a, b)) \propto p(\theta_f) \theta_f^a (1 - \theta_f)^b.$$

the only missing ingredient !

performance prior of
symbolic and prompt programs

symbolic programs [sp]

a program that is written in a symbolic language, and executed with an interpreter with formal execution semantics

- Python
- regular expression
- SQL
- ...

Simple attributes	$\llbracket \tau \vdash x \rrbracket \Downarrow S_x$	s.t. $S_x : \text{env } [\tau] \rightarrow V$
Full attributes	$\llbracket \Gamma \vdash n.x \rrbracket \Downarrow S_{n.x}$	s.t. $S_{n.x} : \text{env } \Gamma \rightarrow V$
Terms	$\llbracket \Gamma \vdash t \rrbracket \Downarrow S_t$	s.t. $S_t : \text{env } \Gamma \rightarrow V$
	$\llbracket \Gamma \vdash \vec{t} \rrbracket \Downarrow S_{\vec{t}}$	s.t. $S_{\vec{t}} : \text{env } \Gamma \rightarrow T \mid \vec{t} \mid$
Queries	$\llbracket \Gamma \vdash_D Q \Rightarrow \tau \rrbracket^{\mathbf{B}} \Downarrow S_Q$	s.t. $S_Q : \text{env } \Gamma \rightarrow R \mid \tau \mid$
Nested queries	$\llbracket \Gamma \vdash_D Q \rrbracket^{\mathbf{B}} \Downarrow S_Q$	s.t. $S_Q : \text{env } \Gamma \rightarrow \text{bool}$
Tables	$\llbracket \Gamma \vdash_D T \Rightarrow \tau \rrbracket^{\mathbf{B}} \Downarrow S_T$	s.t. $S_T : \text{env } \Gamma \rightarrow R \mid \tau \mid$
Frames	$\llbracket \Gamma \vdash_D \overrightarrow{T : \tau} \Rightarrow \Gamma' \rrbracket^{\mathbf{B}} \Downarrow S_{\vec{T}}$	s.t. $S_{\vec{T}} : \text{env } \Gamma \rightarrow R \mid \text{concat } \Gamma' \mid$
Generators	$\llbracket \Gamma \vdash_D G \Rightarrow \Gamma' \rrbracket^{\mathbf{B}} \Downarrow S_G$	s.t. $S_G : \text{env } \Gamma \rightarrow R \mid \text{concat } \Gamma' \mid$
Conditions	$\llbracket \Gamma \vdash_D c \rrbracket^{\mathbf{B}} \Downarrow S_c$	s.t. $S_c : \text{env } \Gamma \rightarrow \mathbf{B}$

has one of these things

prompt programs **[pp]**

task expressed as a prompt (language, examples, etc) that is “executed” by a LLM that output a sequence response directly

- “what is the weather in Singapore last week?”
- “take a look at this clock < img > what time is it?”

we focus on prompt programs with verifiable answers

- e.g. not using LLM to generate pictures, or creative writing, etc.

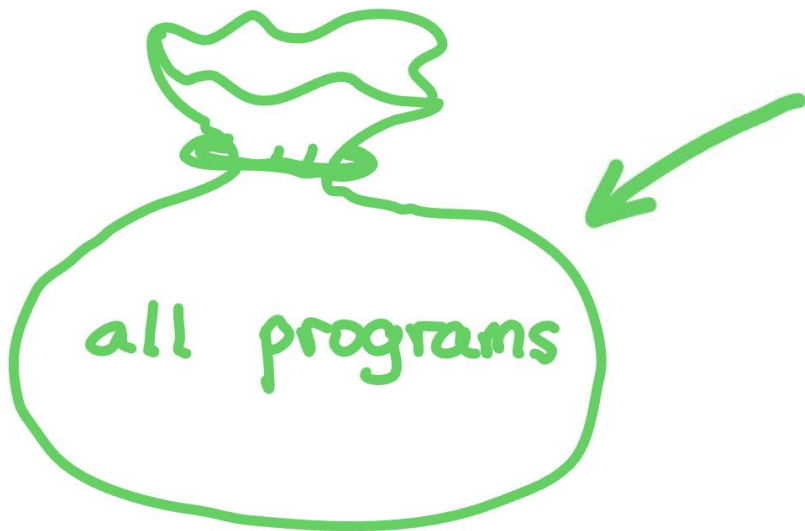
what about asking LLM to generate code ?

what is being delegated for future inputs ?

- the symbolic program is being delegated : a regex is written by a LLM, the regex then operates on thousands of additional strings → **sp**
- the prompt itself is being delegated : a general prompt for solving 12 different programming puzzles in a programming competition → **pp**

getting the prior for sp and pp

ideally: collect every sp and pp ever written



take each program f

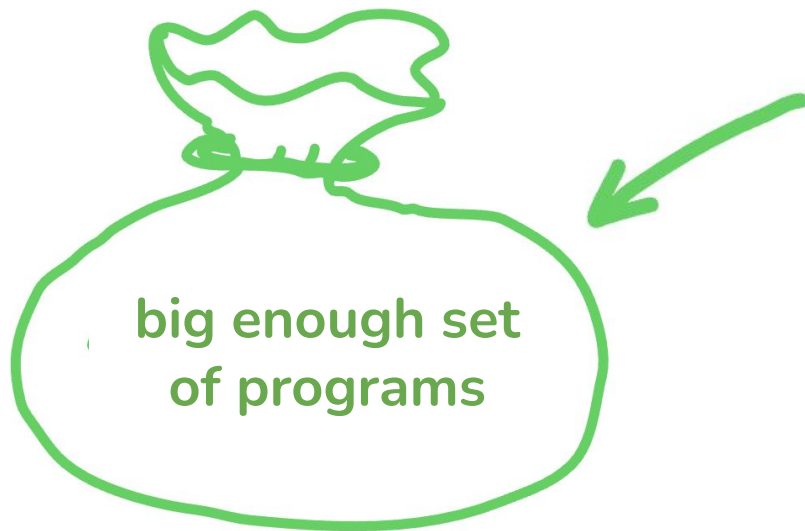
measure its performance θ_f on the task it is written for

- sample 10000 instances from the task and run the program

use the set of measured θ_f as prior

getting the prior for sp and pp

realistically:



take each program f

measure its performance θ_f on the task it is written for

- sample 10000 instances from the task and run the program

use the set of measured θ_f as prior

choosing a set of tasks and their corresponding programs

task consideration:

- each task can be reasonably solved by **both** symbolic programs and prompt programs
- for each task, we have access to many many instances to estimate θ_f

domain	task given as	symbolic program type
Struct. regex	Natural language	Regex expression
HumanEval	Natural language	Python script
ARC	Few shot examples	Python script
List	Few shot examples	Lambda function

choosing a set of tasks and their corresponding programs

task consideration:

- each task can be reasonably solved by **both** symbolic programs and prompt programs
- for each task, we have access to many (600) instances to estimate θ_f

domain	task given as	symbolic program type
Struct. regex	Natural language	Regex expression
HumanEval	Natural language	Python script
ARC	Few shot examples	Python script
List	Few shot examples	Lambda function

in this preliminary work we only considered 35 tasks, will scale later

a task from the regex domain and its solutions

task

natural language description:

A list of five strings. The first string consists of 1+ non-digit characters, the second string consists of 3–4 letters, the third string consists of 4 lowercase letters, the fourth string consists of “LEj” or “uWt”, and the final string consists of “*” or “&”, which is optional.

example input string that match:

-rJeTyItiuWt

sp

```
pattern =  
re.compile(r'^\D+[A-Za-z]{3,4}[a-z]{4}{?:LEj|  
uWt)(?:\*|&)?$')  
  
return bool(pattern.fullmatch(input_string))
```

pp

[[## system message ##]]

You are a master at analyzing strings. I will give you a rule and a string, and you need to analyze whether the string matches the rule, then give me the final answer. Write the answer wrapped by [[## answer ##]], return True or False.

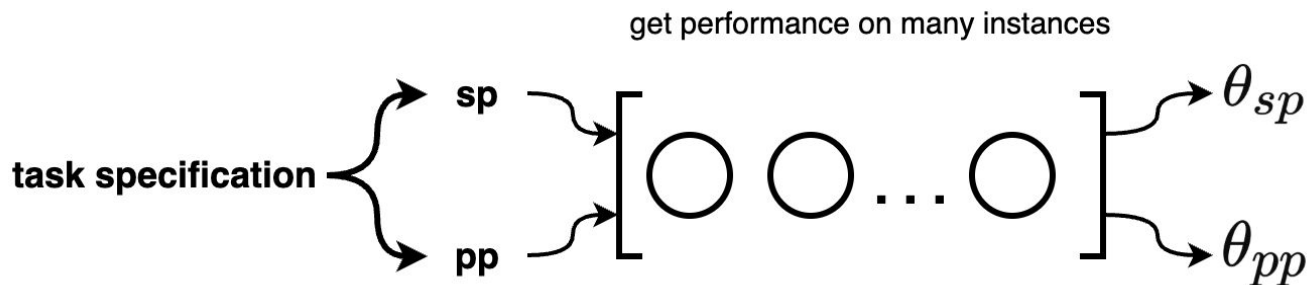
[[## rule ##]]

A list of five strings. The first string consists of 1+ non-digit characters, the second string consists of 3–4 letters, the third string consists of 4 lowercase letters, the fourth string consists of “LEj” or “uWt”, and the final string consists of “*” or “&”, which is optional.

[[## input string ##]]

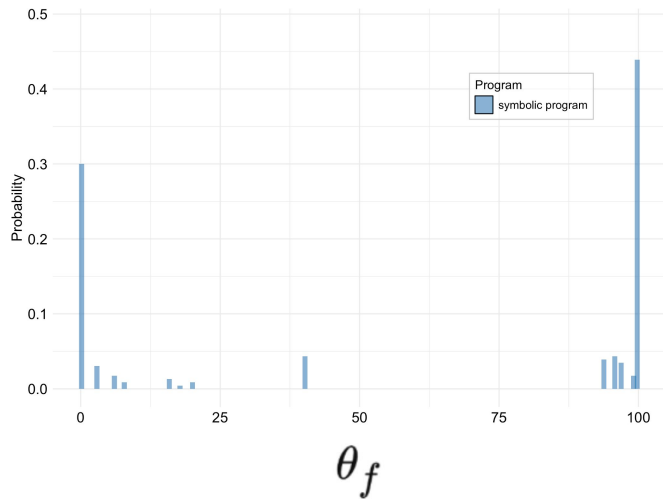
collecting sp and pp performance for all tasks

write multiple sp and pp for each task, and get performance for each sp and pp

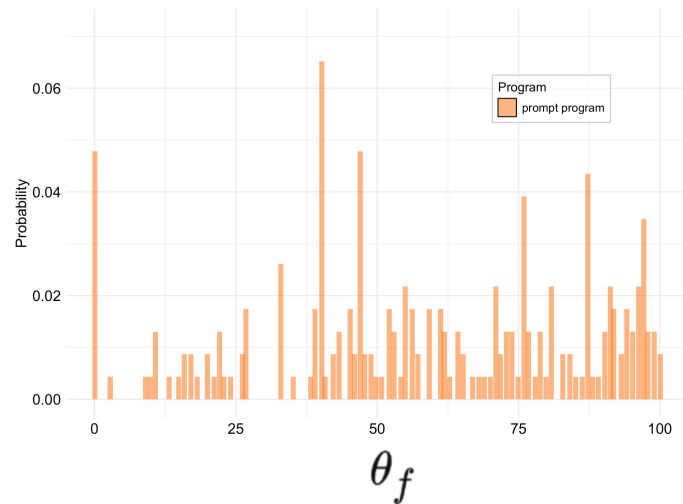


put all the performance number together to form the empirical performance prior

[key slide] empirical prior for sp and pp (350 each)

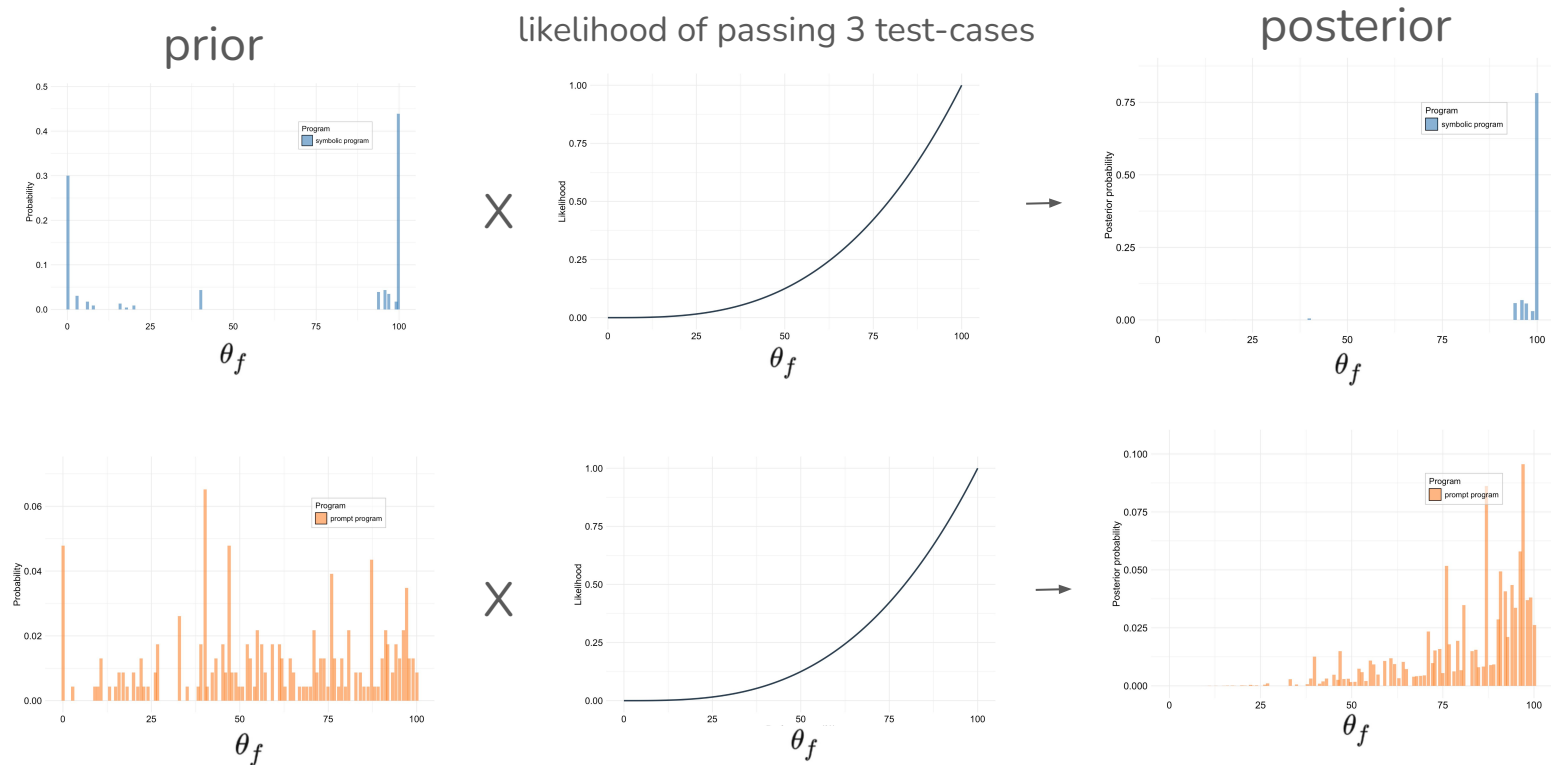


performance prior for sp



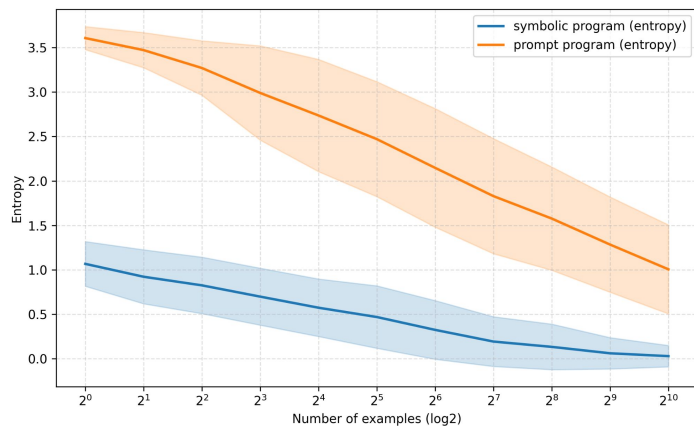
performance prior for pp

performance posterior after passing 3 test-cases

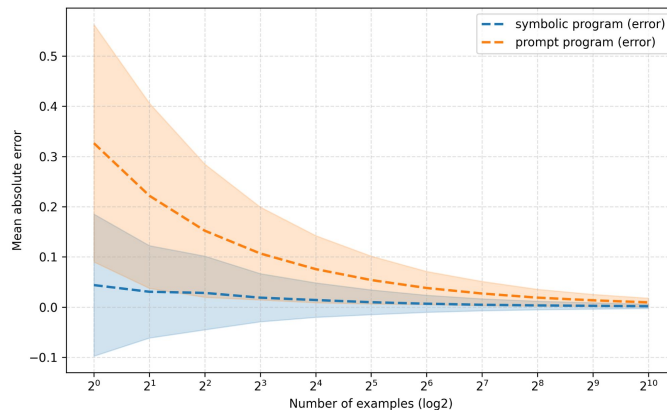


posterior distribution vs number of examples observed

sample a program from the prior, update its posterior with observed examples



entropy



using observed mean as guess
for true performance

conclusion

- prior for sp and pp are very different
- sp prior is bi-modal — all or nothing, thus it is very easy to predict its performance with few examples
- use sp whenever possible
- pp prior is very high in entropy, and not very useful in predicting performance with few examples

performance prediction of
prompt programs from few examples

our plan here

1. take a look at the situation of prompt programs more carefully
2. try to develop an algorithm that leverages a past corpus

prompt programs are general

a single prompt that:

- solves all MBPP
- solves all IMO math problems
- resolves most customer questions

no symbolic program can ever do this

goal: predict performance of pp with few examples

domains and tasks considered

Type	Domain
Knowledge	math
	physics
	chemistry
	economics
	law
Code Generation	engineering
	MBPP
	HumanEval

before:

single **pp** trying to solve many different instances of the same task.

now:

single **pp** trying to solve many different tasks of the same domain

few example tasks and pps

question: Say the pupil of your eye has a diameter of 5 mm and you have a telescope with an aperture of 50 cm. How much more light can the telescope gather than your eye?

choices:

```
[  
  "1000 times more",  
  "50 times more",  
  "5000 times more",  
  "500 times more",  
  "10000 times more",  
  "20000 times more",  
  "2000 times more",  
  "100 times more",  
  "10 times more",  
  "N/A"  
]
```

Your input fields are:

1. `question` (str):
2. `choices` (str):

Your output fields are:

1. `reasoning` (str):
2. `answer` (str): You should output the answer

...

In adhering to this structure, your objective is:

Please read the question carefully in its entirety, paying close attention to key terms and any constraints. Take a moment to think through your reasoning step by step, evaluate each answer choice objectively, and then select the single most appropriate answer based on the information provided.

few example tasks and pps

question: Write a function to find the minimum cost path to reach (m, n) from (0, 0) for the given cost matrix `cost[][]` and a position (m, n) in `cost[][]`.

test_assertion: `assert min_cost([[1, 2, 3], [4, 8, 2], [1, 5, 3]], 2, 2) == 8`

Your input fields are:

1. ``question` (str):`
2. ``test_assertion` (str):`

Your output fields are:

1. ``reasoning` (str):`
2. ``answer` (str):` You should output the complete Python function implementation

All interactions will be structured in the following way, with the appropriate values filled in.

...

In adhering to this structure, your objective is:

You are a professional Python developer. Analyze the natural language description and the assertion test step by step, then write the complete function that meets these specifications.

few example tasks

MMLU PRO

question: Say the pupil of your eye has a diameter of 5 mm and you have a telescope with an aperture of 50 cm. How much more light can the telescope gather than your eye?

choices:

```
[
    "1000 times more",
    "50 times more",
    "5000 times more",
    "500 times more",
    "10000 times more",
    "20000 times more",
    "2000 times more",
    "100 times more",
    "10 times more",
    "N/A"
]
```

Your input fields are:

```
1. 'question' (str):
2. 'choices' (str):
Your output fields are:
1. 'reasoning' (str):
2. 'answer' (str): You should output the answer letter
All interactions will be structured in the following way, with the appropriate values filled in.
```

```
[[ ## question ## ]]
{question}
```

```
[[ ## choices ## ]]
{choices}
```

```
[[ ## reasoning ## ]]
{reasoning}
```

```
[[ ## answer ## ]]
{answer}
```

```
[[ ## completed ## ]]
```

In adhering to this structure, your objective is:

Please read the question carefully in its entirety, paying close attention to key terms and any constraints. Take a moment to think through your reasoning step by step, evaluate each answer choice objectively, and then select the single most appropriate answer based on the information provided.

question: Colors in a soap bubble result from light

choices:

```
[
    "dispersion",
    "deflection",
    "refraction",
    "reflection",
    "interference",
    "converted to a different frequency",
    "polarization",
    "absorption",
    "diffraction",
    "transmission"
]
```

Your input fields are:

```
1. 'question' (str):
2. 'choices' (str):
Your output fields are:
1. 'reasoning' (str):
2. 'answer' (str): You should output the answer letter
All interactions will be structured in the following way, with the appropriate values filled in.
```

```
[[ ## question ## ]]
{question}
```

```
[[ ## choices ## ]]
{choices}
```

```
[[ ## reasoning ## ]]
{reasoning}
```

```
[[ ## answer ## ]]
{answer}
```

```
[[ ## completed ## ]]
```

In adhering to this structure, your objective is:

You are an expert across a wide range of subjects. Please read the following question carefully, consider all relevant details and potential interpretations, and then select the single best answer. Prioritize accuracy and sound reasoning over speed, and ensure your final choice is the most defensible option based on the information provided.

MBPP

question: Write a function to find the minimum cost path to reach (m, n) from (0, 0) for the given cost matrix cost[][] and a position (m, n) in cost[][].

test_assertion: assert min_cost([[1, 2, 3], [4, 8, 2], [1, 5, 3]], 2, 2) == 8

Your input fields are:

```
1. 'question' (str):
2. 'test_assertion' (str):
Your output fields are:
1. 'reasoning' (str):
2. 'answer' (str): You should output the complete Python function implementation
All interactions will be structured in the following way, with the appropriate values filled in.
```

```
[[ ## question ## ]]
{question}
```

```
[[ ## test_assertion ## ]]
{test_assertion}
```

```
[[ ## reasoning ## ]]
{reasoning}
```

```
[[ ## answer ## ]]
{answer}
```

```
[[ ## completed ## ]]
```

In adhering to this structure, your objective is:

You are a professional Python developer. Analyze the natural language description and the assertion test step by step, then write the complete function that meets these specifications.

question: Write a function to find the top k integers that occur most frequently from given lists of sorted and distinct integers using heap queue algorithm.

test_assertion: assert funcn([[1, 2, 6], [1, 3, 4, 5, 7, 8], [1, 3, 5, 6, 8, 9], [2, 5, 7, 11], [1, 4, 7, 8, 12]],3)=[5, 7, 1]

Your input fields are:

```
1. 'question' (str):
2. 'test_assertion' (str):
Your output fields are:
1. 'answer' (str): You should output the complete Python function implementation
All interactions will be structured in the following way, with the appropriate values filled in.
```

```
[[ ## question ## ]]
{question}
```

```
[[ ## test_assertion ## ]]
{test_assertion}
```

```
[[ ## answer ## ]]
{answer}
```

```
[[ ## completed ## ]]
```

In adhering to this structure, your objective is:

Please write a Python function according to the given specification and test case. Ensure your implementation passes the provided assertion.

the boundary of domains are blurry

business domain:

Typical advertising regulatory bodies suggest, for example that adverts must not: encourage _____, cause unnecessary _____ or _____, and must not cause _____ offence.

choices:

```
[  
"Safe practices, Fear, Jealousy, Trivial",  
"Unsafe practices, Distress, Joy, Trivial",  
"Safe practices, Wants, Jealousy, Trivial",  
...  
"Unsafe practices, Wants, Jealousy, Serious",  
"Unsafe practices, Wants, Fear, Trivial",  
]
```

math domain:

If the parabola $y_1 = x^2 + 2x + 7$ and the line $y_2 = 6x + b$ intersect at only one point, what is the value of b ?

choices:

[10,5,3,12,4,6,8,7,2,9]

a single **pp** prompted for multiple-choice, think step-by-step can solve both tasks

more words on domain boundaries

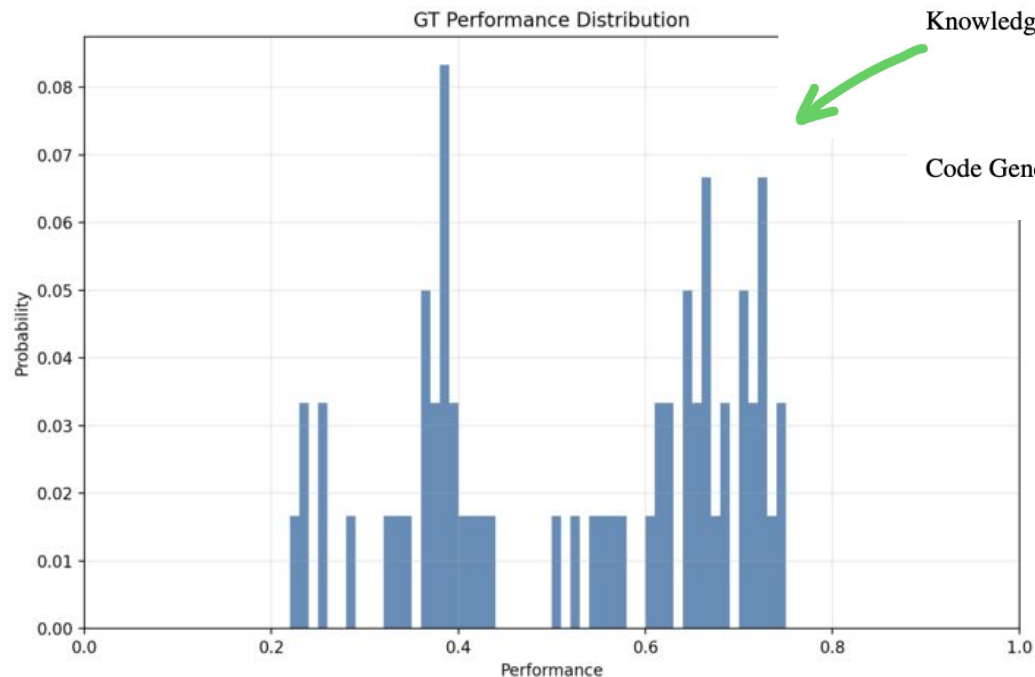
for symbolic programs **sp** :

- 1 DSL or framework per entire domain
- framework will not generalize to a different domain

for prompt programs **pp** :

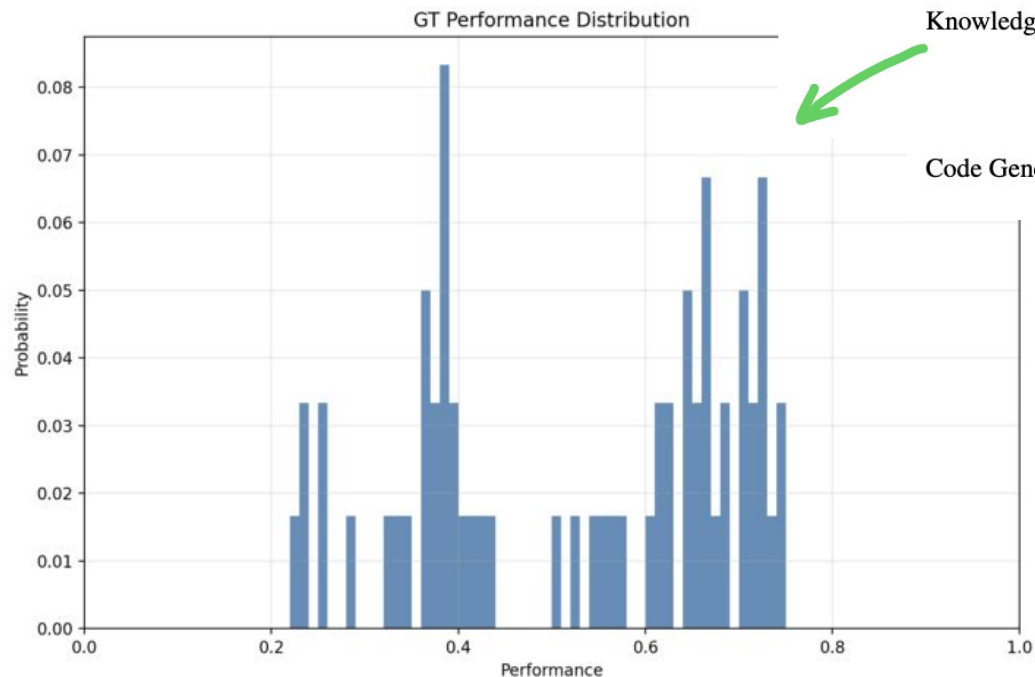
- one prompt can work for multiple domains
- tasks similarities is more important than “domain” boundaries

prior of all pps is high entropy



Type	Domain
Knowledge	math
	physics
	chemistry
	economics
	law
Code Generation	engineering
	MBPP
	HumanEval

prior of all pps is high entropy



Type	Domain
Knowledge	math
	physics
	chemistry
	economics
	law
Code Generation	engineering
	MBPP
	HumanEval

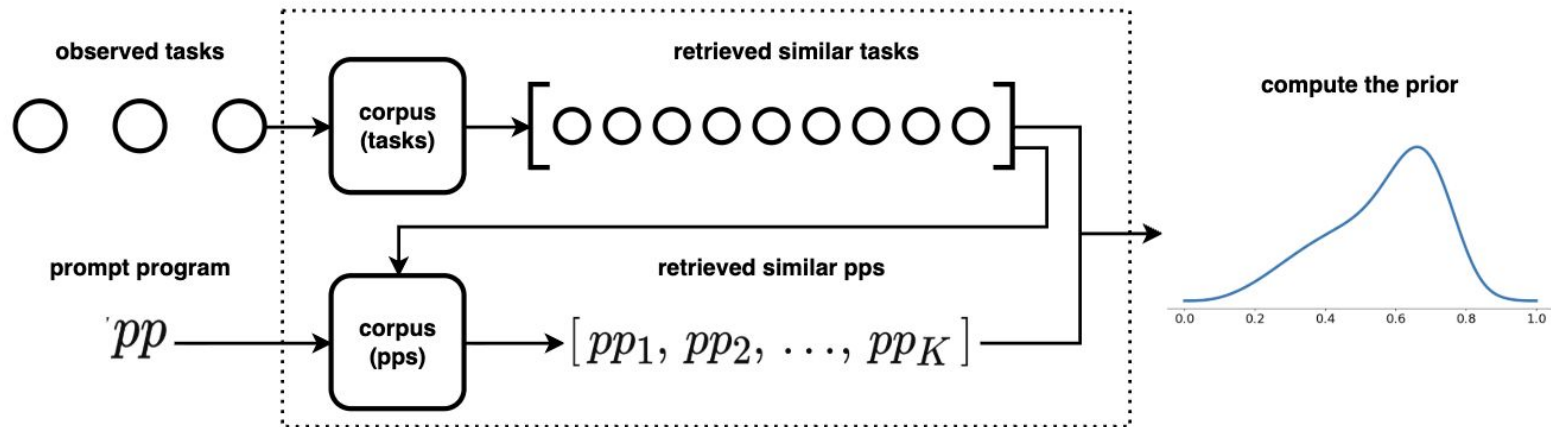
this makes predictions from examples difficult

can we build a more “focused” prior

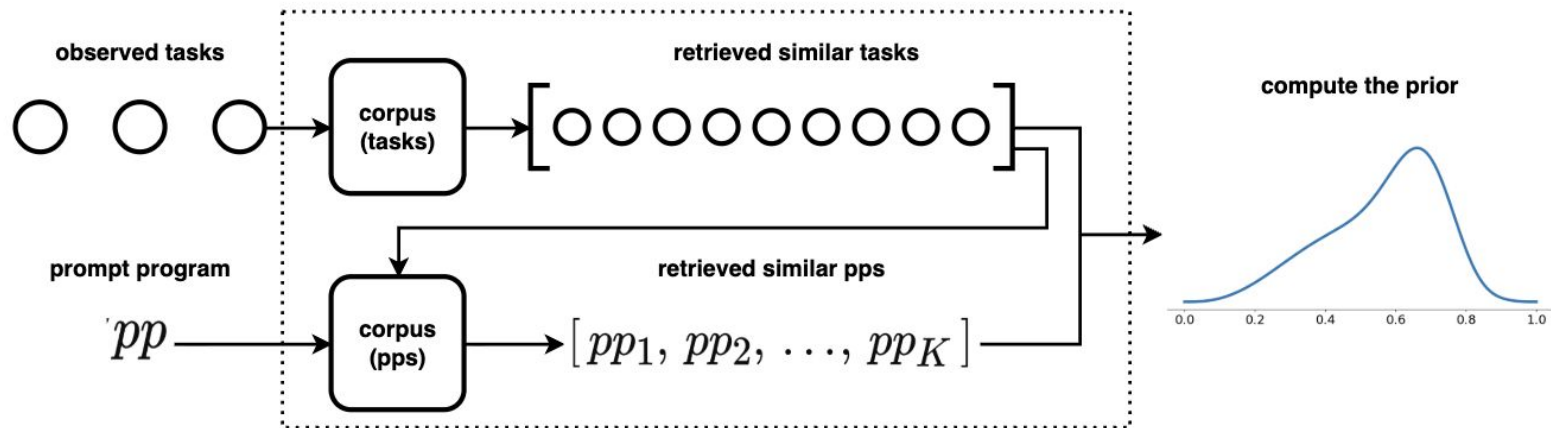
- build this prior from similar tasks and similar pps
- cross domain “boundaries”

key idea: retrieve similar tasks and pps

a pretty reasonable algorithm based on retrieval

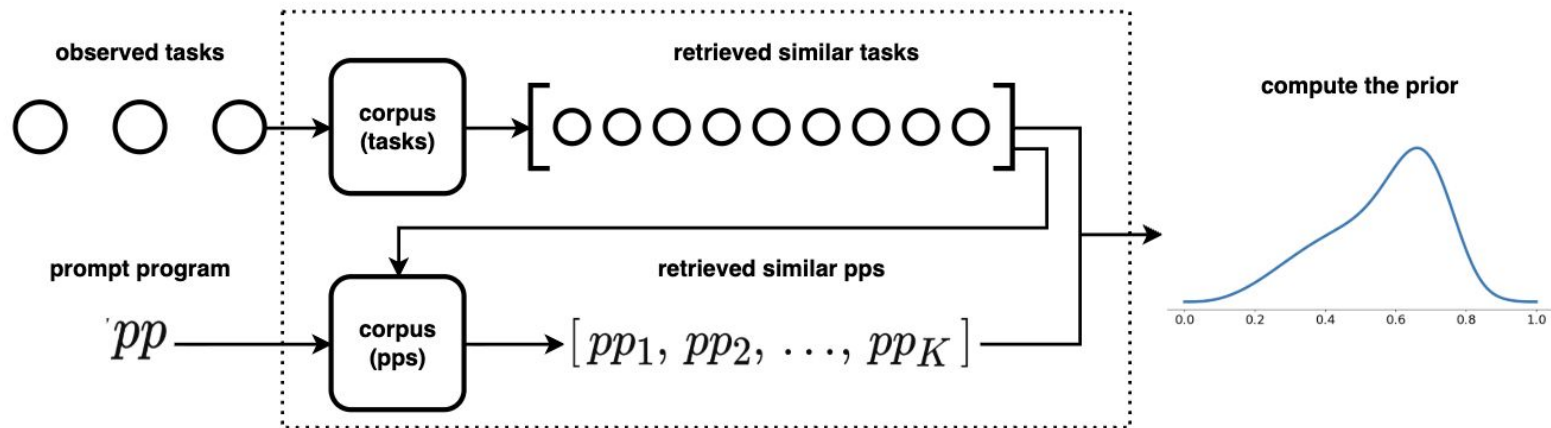


a pretty reasonable algorithm based on retrieval



the performance of the retrieved pp forms a support, and we build a mixture of Beta distributions on top for smoothing

a pretty reasonable algorithm based on retrieval



similar tasks $\leftarrow$ text embedding similarity

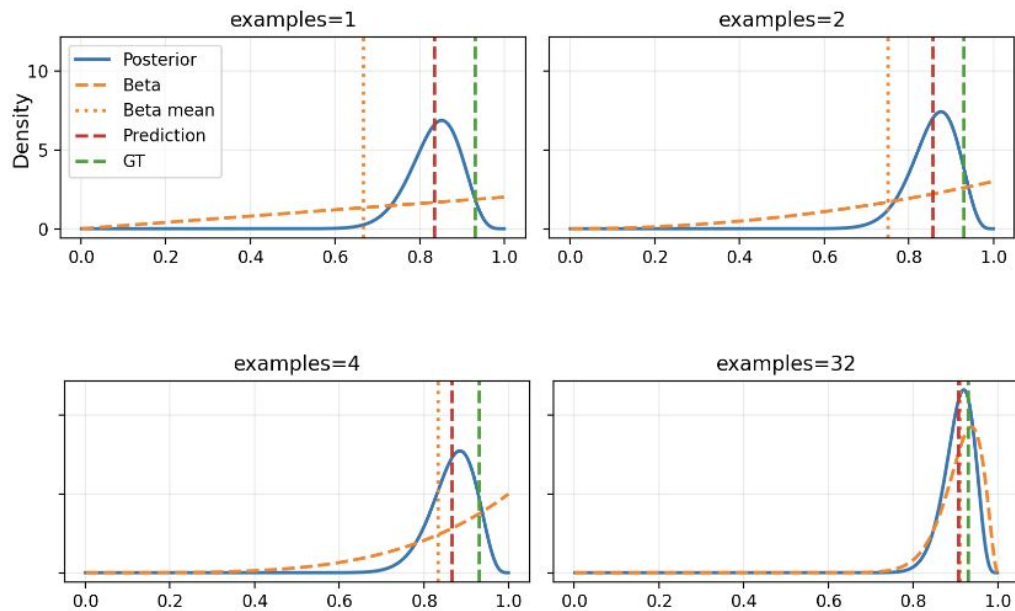
similar pps $\leftarrow$ if they solve similar subset of retrieved tasks

no use of “domain category” (math, biology, ...)

no use of “structure” of pp (prompt style, gpt-4 or qwen, ...)

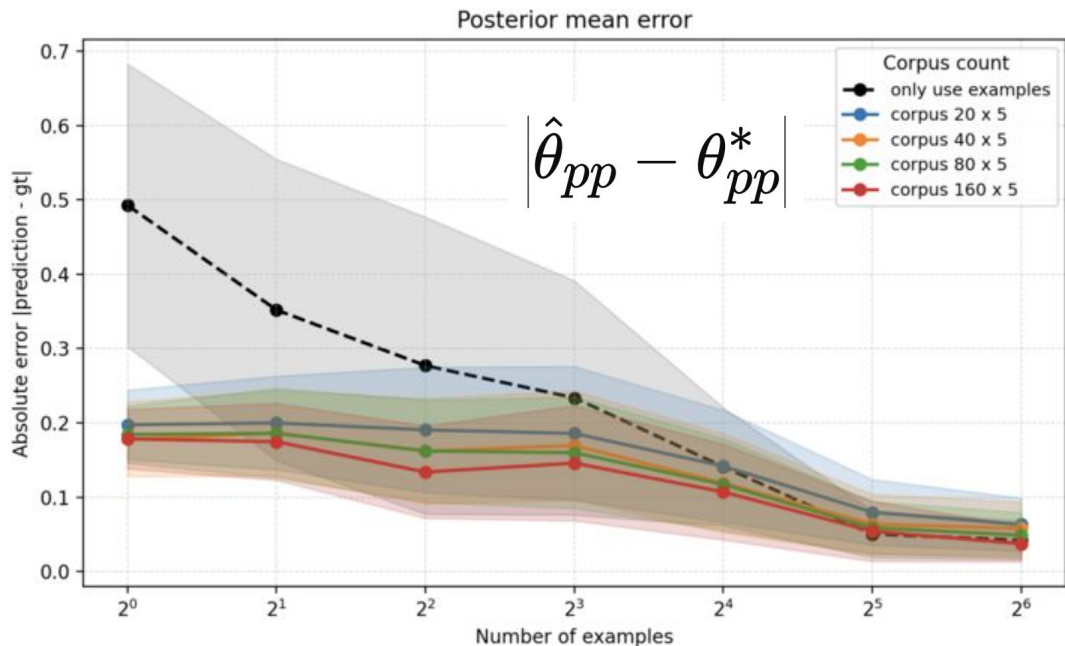
(very) preliminary results !

in predicting the value θ_f



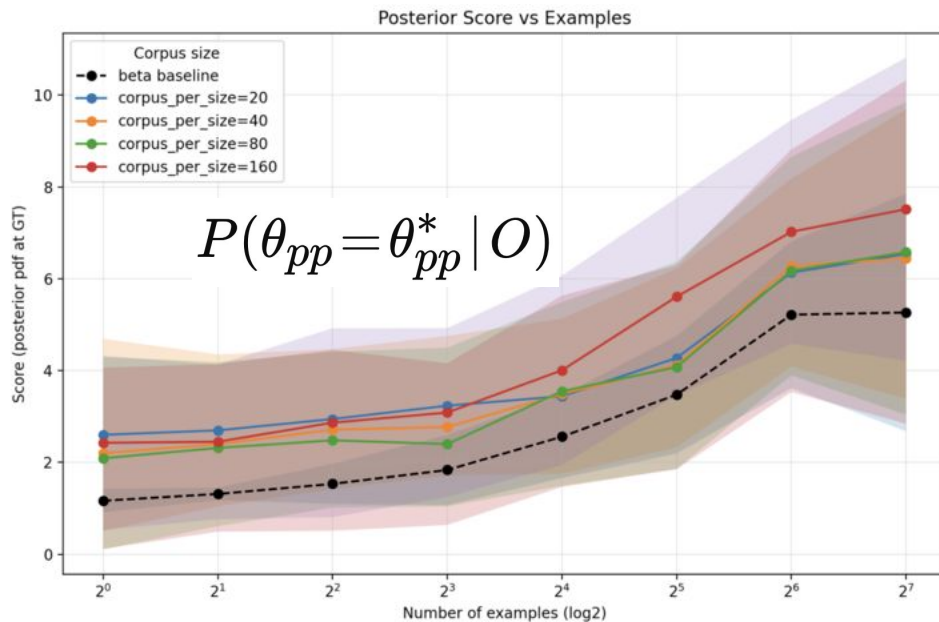
(very) preliminary results !

in predicting the value θ_f



(very) preliminary results !

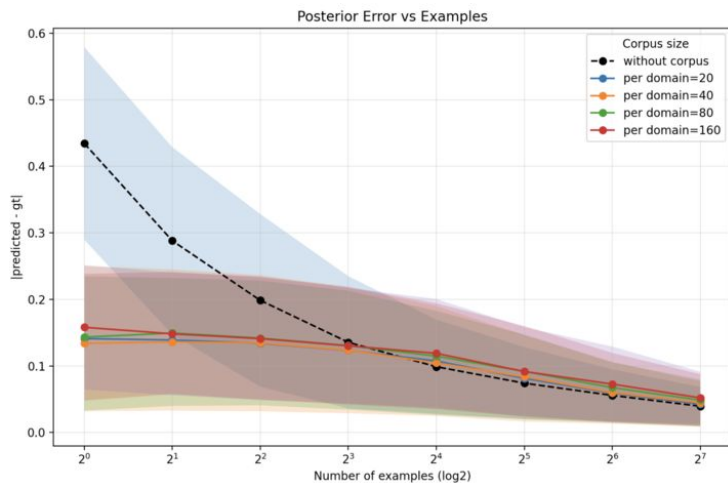
probability of ground-truth under prediction



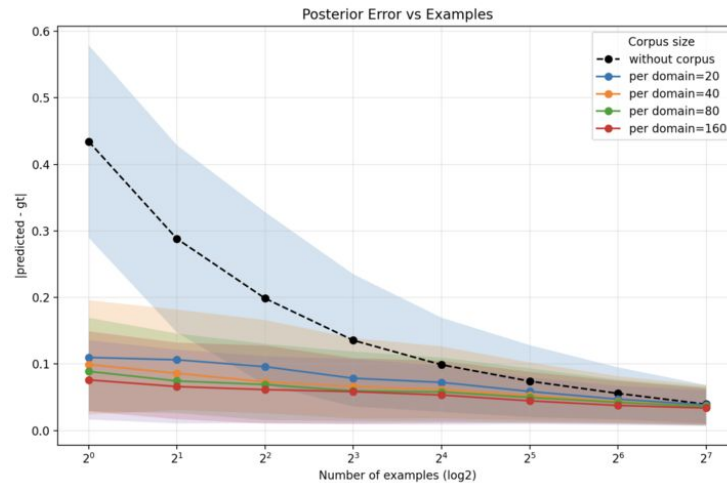
(very) preliminary results !

If the corpus contain very similar tasks, it performs well

corpus without in-domain tasks



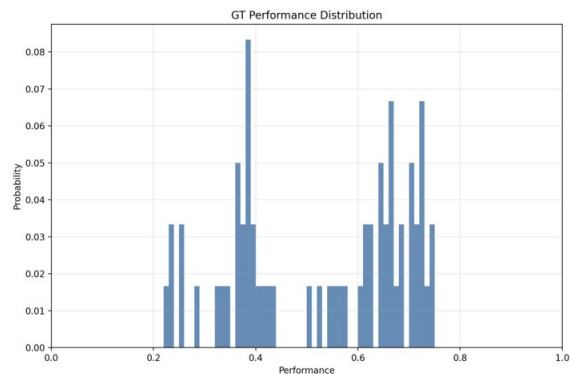
corpus with in-domain tasks



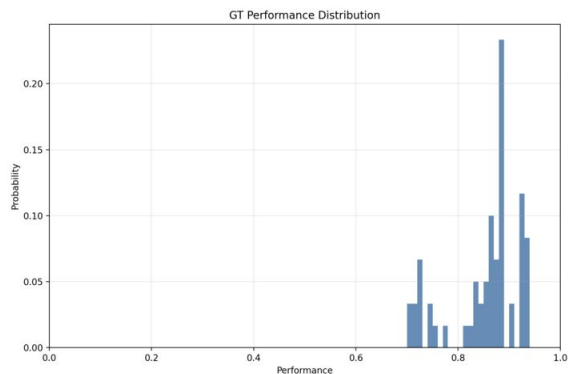
when our algorithm doesn't work

when **pp** already behaves like **sp**, few examples will give an accurate estimate

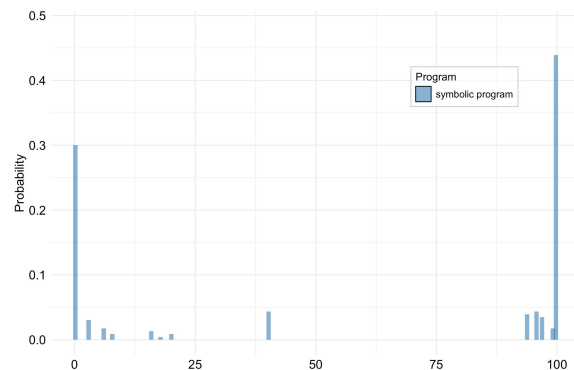
- strong LLMs on simpler tasks behaves more like **sp**
- caveat: still not 100% ← so be careful! (open challenge)



4o-mini



5o-mini



symbolic programs
(from before)

thanks ! please give a lot of feedback haha

